

BendTT: An Affine Dependent Type Theory

Victor Taelin
Higher Order Company
Rio de Janeiro, Brazil
taelin@higherorderco.com

AI DISCLOSURE. Bend and BendTT were designed by the human author. This paper was written by Claude Fable 5, based on the author’s code and design choices, and thoroughly reviewed by the author. The Lean formalization was human-specified, AI-proven, and checked by Lean.

Abstract

BendTT is the proof kernel of the Bend programming language: a dependent type theory with a single sort satisfying $\text{Type} : \text{Type}$, no universe hierarchy, and no positivity restriction on its recursive datatypes. It is nevertheless consistent, because the calculus is completely affine: no live value may be consumed more than once, duplication of any kind is unwritable in the kernel, and every classical paradox needs to duplicate a function at some point. Recursion is checked by a lexicographic strict-subterm descent over each definition’s own case tree, and only the live level must terminate: dead code and types may diverge. Duplication returns at the surface as a license the program itself proves, a copying function with equality proofs, never as a kernel rule. Everything else is standard, by design. We define the calculus, argue informally that it terminates, and describe a Lean 4 mechanization of its metatheory: confluence, subject reduction, progress, weak normalization, and consistency, with no unproven obligations. This is a working document and will evolve with the language.

1 Introduction

BendTT is the proof kernel of Bend, a programming language built around dependent types, affine values, and a parallel runtime. This paper defines the kernel, explains why it is consistent, and points to the mechanized proof.

The paper contributes no new machinery to type theory, and is not meant to. Its purpose is to document the theory behind Bend, as a reference for users and auditors. Nearly every ingredient is deliberately standard: user-declared datatype families with dependent elimination; plain small-step reduction; intensional equality with rewriting. The exception, and the only part a type theorist may find surprising, is that BendTT admits $\text{Type} : \text{Type}$ and unrestricted recursive types, negative occurrences included, yet remains consistent. The reason is affinity: a value can never be used more than once, and the engines behind the classical paradoxes all duplicate a $\forall$ -typed value at some point, so none of them typechecks. That contraction, not impredicativity, powers the paradoxes is an old observation. Grišin proved naive set comprehension consistent over a logic without contraction [26], and Girard and Terui rebuilt naive set theories over light and affine linear logics [24, 44]. BendTT transplants that principle into a dependent type theory,

where it replaces the universe hierarchy outright, and the result is mechanized in Lean 4 (Section 5). An earlier version of this kernel carried a built-in judgment classifying the types whose values may be copied; the present kernel goes further and admits *no* duplication at all: the copying permit moved out of the kernel and into the programs, as a proof obligation (Section 4).

The implemented language is larger than the kernel: machine words and floats, flat arrays, and the surface copying license itself exist in the implementation and are not yet formalized (Section 6); those gaps are listed exactly. The calculus also deliberately leaves room for extensions such as quotients and extensionality principles, which require a redesign of propositional equality.

For context, the language this kernel serves, in one paragraph. Bend is a Python-shaped functional language with dependent types. A declaration is an `assert`, the full type, plus a `def`, its fill; `match/case` blocks over top-level parametric datatypes flatten into case trees, machine 32-bit words and floats and flat arrays ride beside them, propositional equality rewrites through explicit motives, and every definition passes the descent check. Values are affine by default; a binder marked `+` may be consumed freely, and its type must carry the proved copying license of Section 4. One program hint, the fork `let`, drives a bulk-synchronous runtime: programs compile to a standalone C file that runs on CPU threads and on Apple-Silicon GPUs [42]. The checker is bidirectional, with no unification, no metavariables, and no implicit arguments anywhere. Excluding the parser and the printer, the reference checker is a few thousand lines of TypeScript.

2 The Calculus

The calculus is a handful of definitions: terms and books, reduction, quantities, typing, and the descent rule for recursion. The first two are nearly conventional; the unconventional points are flagged as they appear.

2.1 Terms and Books

Figure 1 gives the term formers. Types are terms, and any term may appear anywhere. Four remarks.

First, datatypes are primitive, and they are the programmer’s own. A *book* carries, beside its definitions, a list of datatype families: a family signature is a telescope of parameters tipped at Type , and each constructor is a tele-

$t, A, B ::=$	
x	variable
$@k$	reference to definition k
Type	the sort
$\forall^q x : A. B$	function type
$\lambda x. t$	abstraction
$t u$	application
$q x = t; u$	let (not recursive)
$D^r p_1 \dots p_n$	datatype family, applied
$C\{t_1, \dots, t_n\}$	constructor of a family
$\lambda\{C : h; m\}$	match: peel one constructor
$\lambda\{\}$	empty match
$\{a = b : T\}$	propositional equality
$\{=\}$	reflexivity
$\% e : P; f$	rewrite by e through motive P

Figure 1. Terms. One syntactic category: types are terms. q ranges over the quantities of Section 2.3. A family head D^r carries the set r of constructors a match has already peeled, empty when written; constructors and families are *heads*, saturated by ordinary application.

scope of erased parameters then fields, tipped at the family applied to exactly its own parameters, in order. There is no positivity restriction: a field may mention the family to the left of an arrow, and $\text{Trm} : \text{Lam}\{f : \text{Trm} \rightarrow^1 \text{Trm}\}$ is a legal declaration. Section 3 explains why that is harmless. There is also no elimination principle to generate: the eliminator is the match.

Second, the match is a *peel*, not a table: $\lambda\{C : h; m\}$ applied to a C -headed value passes the constructor's fields to h , and any other constructor falls through to the default m , whose domain records C in the family's peeled set r , which conversion ignores. A full case split is a chain of peels ending at the empty match $\lambda\{\}$, which eliminates a family with every constructor peeled; the implementation's *match/case* blocks flatten to exactly this shape. Eliminators are ordinary values that consume their scrutinee by application, so a definition written as nested lambdas and matches is a case tree, the shape the descent rule of Section 2.5 walks.

Third, the rewrite $\% e : P; f$ carries its motive P in the term, a two-binder function over the carrier and the equation. This is the J axiom and nothing more: from $e : \{a = b : T\}$, the rewrite maps a goal $P b e$ to an obligation $P a \{=\}$. The motive is written by the programmer, not synthesized: Bend has no unification, so wherever a motive is needed, the source states it (Section 6). Equality here is the quiet, old-fashioned kind, on purpose: intensional, computationally irrelevant, with no function extensionality and no higher-dimensional structure; anything fancier is deferred (Section 8).

Fourth, a book is *ordered*, and a definition is two events. An assert declares a closed type; until its fill arrives, the name is an *axiom*: it may be referenced in dead positions but never consumed live, so an axiom can state anything and prove nothing. A def fills a prior assert with a closed body, checked live against the asserted type. Definition k may reference $@j$ for $j < k$ freely, and itself only through the descent rule of Section 2.5; a reference to a *later* name

$$\begin{aligned}
(\lambda x. f) a &\rightarrow f[x := a] \\
q x = v; b &\rightarrow b[x := v] \\
@k a_1 \dots a_{n_k} &\rightarrow \beta(k) a_1 \dots a_{n_k} \\
@D &\rightarrow D^\emptyset \\
\lambda\{C : h; m\} (C\{p_1 \dots, x_1 \dots\}) &\rightarrow h x_1 \dots x_n \\
\lambda\{C : h; m\} (C'\{\dots\}) &\rightarrow m (C'\{\dots\}) \\
\% \{=\} : P; f &\rightarrow f \\
\lambda x. F x \quad (x \notin F) &\rightarrow F
\end{aligned}$$

Figure 2. Reduction. n_k is definition k 's declared arity; the $@k$ rule fires at weak strength only when the spine is saturated, at strong strength at any arity, and the η -contraction is strong-only. The congruent closure comes at two strengths: *strong* reduction may enter any subterm, *weak* reduction never enters a binder. Conversion $a \simeq b$ is joinability under strong reduction.

is refused outright, so the reference graph is acyclic by construction and mutual recursion cannot smuggle a loop past the wall. References carry code, not resources: $@k$ costs nothing in the usage accounting below.

2.2 Reduction

Figure 2 lists the interactions. A reference unfolds only at saturation and only with a body: an underapplied or unfilled reference spine is a stuck weak-head value, which is what makes an axiom inert and lets the checker keep goals in the vocabulary of the source. At strong strength, conversion's reach, a reference may unfold at any arity, and η -contraction applies, so a saturating argument can unfold what an underapplied definition kept closed and conversion is up to η for functions. A bare nullary family reference steps to its family head; a *parameterized* family head is deliberately stuck and has no bare spelling: $D\langle p_1, \dots \rangle$ is the one way to write it. A match meeting its constructor takes the fields (the erased parameters p_i are skipped); any other constructor falls to the default, whole. A rewrite sheds when its evidence reaches $\{=\}$ for closed live terms, canonicity guarantees it does. Values are the type formers, constructor- and family-headed spines, function values, and stuck reference spines.

There is nothing else: no side conditions, no strategy. Affinity is enforced by the typing judgment, which counts usage at check time (Section 2.4); reduction itself never blocks. Which claims hold at which strength is stated exactly in Section 2.6.

2.3 Quantities and Usage

Binders carry a quantity $q \in \{0, 1\}$: erased or affine, spelled $-x$ and x in the surface. Throughout, *affine* means dropping is free and duplicating does not exist. A third value ω exists only inside the usage accounting, where it means "consumed more than once" and is always a violation: no rule of the kernel forms an ω binder, and the surface spelling $+x$ belongs to the license of Section 4, not to the kernel.

Usage is tracked by the judgment itself. A derivation returns a *usage vector* π , a finitely supported map from variables to quantities, recording what the term consumed. We write $\emptyset$ for the empty vector, x^q for the singleton, $\pi + \pi'$ for the pointwise saturating sum ($1 + 1 = \omega$), $\pi \sqcup \pi'$ for

$$\begin{array}{c}
\frac{(x : T) \in \Gamma}{\Gamma \vdash_q x : T \triangleright x^q} \quad \frac{\beta(k) = (T, t) \quad q \neq 0 \Rightarrow t \neq \text{unfilled}}{\Gamma \vdash_q @k : T \triangleright \emptyset} \quad \frac{}{\Gamma \vdash_q \text{Type} : \text{Type} \triangleright \emptyset} \quad \frac{\beta(D) \text{ a nullary family}}{\Gamma \vdash_q @D : \text{Type} \triangleright \emptyset} \\
\\
\frac{\beta(D) \text{ a family}}{\Gamma \vdash_q D^r : \text{sig}(D) \triangleright \emptyset} \quad \frac{C \text{ a constructor} \quad C \notin r}{\Gamma \vdash_q C : \text{tele}(C, r) \triangleright \emptyset} \quad \frac{q' \neq \omega \quad \Gamma \vdash_0 A : \text{Type} \triangleright \cdot \quad \Gamma, x : A \vdash_0 B : \text{Type} \triangleright \cdot}{\Gamma \vdash_q \forall^{q'} x : A. B : \text{Type} \triangleright \emptyset} \\
\\
\frac{\Gamma, x : A \vdash_q f : B \triangleright \pi \quad \pi(x) \leq q'}{\Gamma \vdash_q \lambda x. f : \forall^{q'} x : A. B \triangleright \pi \setminus x} \quad \frac{q' \neq \omega \quad \Gamma \vdash_q f : \forall^{q'} x : A. B \triangleright \pi \quad \Gamma \vdash_{q' \triangleright q} a : A \triangleright \pi'}{\Gamma \vdash_q f a : B[x := a] \triangleright \pi + \pi'} \\
\\
\frac{q_b \neq \omega \quad \Gamma \vdash_{q_b \triangleright q} v : A \triangleright \pi' \quad \Gamma, x : A \vdash_q b : B \triangleright \pi \quad \pi(x) \leq q_b \quad x \notin B}{\Gamma \vdash_q q_b x = v; \quad b : B \triangleright \pi' + (\pi \setminus x)} \\
\\
\frac{\Gamma \vdash_0 T : \text{Type} \triangleright \cdot \quad \Gamma \vdash_0 a : T \triangleright \cdot \quad \Gamma \vdash_0 b : T \triangleright \cdot}{\Gamma \vdash_q \{a = b : T\} : \text{Type} \triangleright \emptyset} \quad \frac{a \simeq b}{\Gamma \vdash_q \{=\} : \{a = b : T\} \triangleright \emptyset} \\
\\
\frac{\Gamma \vdash_q e : \{a = b : T\} \triangleright \pi \quad \Gamma \vdash_0 P : \forall^0 x : T. \{a = x : T\} \rightarrow^1 \text{Type} \triangleright \cdot \quad \Gamma \vdash_q f : P a \{=\} \triangleright \pi'}{\Gamma \vdash_q \% e : P; \quad f : P b e \triangleright \pi + \pi'} \\
\\
\frac{C \notin r \quad q \neq 0 \Rightarrow q' \neq 0 \quad \Gamma \vdash_q h : \text{arms}(C, p_1 \dots, q', B) \triangleright \pi \quad \Gamma \vdash_q m : \forall^{q'} (D^{r \cup \{C\}} p_1 \dots). B \triangleright \pi'}{\Gamma \vdash_q \lambda \{C : h; \quad m\} : \forall^{q'} (D^r p_1 \dots). B \triangleright \pi \sqcup \pi'} \\
\\
\frac{\text{every } C \text{ of } D \in r \quad q \neq 0 \Rightarrow q' \neq 0}{\Gamma \vdash_q \lambda \{\} : \forall^{q'} (D^r p_1 \dots). B \triangleright \emptyset} \quad \frac{\Gamma \vdash_q t : A \triangleright \pi \quad A \simeq B}{\Gamma \vdash_q t : B \triangleright \pi}
\end{array}$$

Figure 3. Typing. The judgment $\Gamma \vdash_q t : T \triangleright \pi$ reads: under book β (left implicit) and context Γ , at demand $q \in \{0, 1\}$, term t has type T consuming π . $q' \triangleright q$ is the gate of Section 2.3 (demand 0 when $q' = 0$, else q). In REF, the side condition is that a *live* reference needs a filled body: an axiom types only at demand 0. In CTR, $\text{tele}(C, r)$ is constructor C 's declared telescope, its tip's family head re-annotated with any peeled set r not containing C , which is what lets a scrutinee re-check at the peeled domain of a match's default. In MAT, $\text{arms}(C, p_1 \dots, q', B)$ is C 's field telescope with the parameters instantiated at $p_1 \dots$, each field's quantity gated by q' , tipped at B applied to the constructor rebuilt from the fields. Conversion in side conditions is relative to β .

the pointwise join (match arms: one branch runs, so arms share), and $\pi \setminus x$ for removal. Quantities are ordered $0 \leq 1 \leq \omega$, and a binder of quantity q admits any measured usage $\leq q$. There is no scaling operation and no multiplication anywhere: an argument's demand is *gated*, not multiplied, by the function's domain quantity, written $q' \triangleright q$, which is 0 when the domain is erased and the ambient demand q otherwise.

2.4 Typing

Figure 3 gives the rules. The judgment $\Gamma \vdash_q t : T \triangleright \pi$ carries a *demand* q : the quantity at which the term is being consumed. Most of the figure is what any reader would guess; the content is in the following five points.

The wall is total. There is no rule that forms an ω binder: the function-type rule and the let rule refuse the quantity outright, so contraction is not restricted, it is unwritable. A binder's measured usage must fit its declared quantity when it closes ($\pi(x) \leq q'$), and two live uses saturate the measure to ω , which no binder satisfies. This single check is the consistency mechanism (Section 3); the implemented language re-admits contraction only through the surface license of Section 4, which the kernel never sees as a rule.

The erased fragment is free. At demand 0 nothing is charged: a dead premise's measure is unconstrained and the rule that checks it dead drops it from the conclusion (written

in the figure). Types in premises, equality endpoints, motives, and erased arguments all check dead. Erased code is specification, not proof; it can even inhabit an empty family (Section 2.6), and no rule coerces dead to live.

Matching is consumption. In a live region a match's domain quantity must be live ($q \neq 0 \Rightarrow q' \neq 0$): an erased scrutinee cannot be inspected by running code. The arm receives the constructor's fields at quantities gated by the scrutinee's, the default receives the scrutinee with the handled constructor peeled onto the family head, and the two usages *join* rather than add, since one branch runs. The empty match closes a family whose every constructor is peeled.

Equality is dead where it states and live where it runs. The equation's components live in the type and are checked erased, so a proof may mention already-consumed variables there for free, which is the idiom inductive proofs live on. The evidence of a rewrite, by contrast, runs (to $\{=\}$), so it is checked at the ambient demand; the motive is checked dead.

Conversion, and no totality. The conversion rule moves the goal along $\simeq$ and is the only mode switch. Nothing makes checking total: conversion may diverge on dead code, and the checker is a semi-decision procedure. This is deliberate; see Section 3.3.

2.5 Descent: the Recursion Rule

Self-reference passes one test, run on the definition’s own case tree. Walking the body, the checker rebuilds the definition’s left-hand side: a lambda binds the next *column*, a match peels the current column into a constructor of fresh field columns, and at every leaf each self-reference must head a whole call whose arguments *descend* against the columns: reading left to right and skipping erased columns, every argument compares equal to its column until one is a *strict subterm* of it, the same constructor with at least one strictly smaller field, or a term sitting inside one of the pattern’s fields, one constructor peel deep. Arguments past the decreasing one are unconstrained. No sizes are computed and no size-change graph exists: the order is syntactic, the comparison sees through lets and annotations, and the whole check is one pass over the elaborated body. Erased columns are skipped because erased data cannot be matched by live code, so it can never carry the decrease.

Concretely:

```
assert add:
  forall n: Nat
  forall m: Nat
  Nat

def add(n, m):
  match n:
  case 0n:
    m
  case 1n+p:
    1n + add(p, m)
```

The successor arm binds p one peel inside n , so the self-call’s first column is strictly smaller and the second is never consulted. A lexicographic pair descends the same way: $\text{ack}(1n+i, 0n)$ may call $\text{ack}(i, 1n)$ (first column smaller, second free) and $\text{ack}(1n+i, 1n+j)$ may call $\text{ack}(1n+i, j)$ (first equal, second smaller). To be clear about the ambition: this is a minimal member of the structural-recursion family [1, 22, 31], chosen because it is trivial to audit, and it is the rule the implementation runs, not an idealization of it.

2.6 Books, and the Claims

A book β is *well-formed* (Ok) when, in the empty context: every family signature and constructor telescope checks dead against Type and has its declared shape, each constructor tipping at its own family; every definition’s type checks dead against Type every filled body checks *live* against its type; and every body’s case tree descends, with no reference at or beyond its own index except the guarded self-call. There is no separate termination checker bolted onto the language: Ok is the language. The metatheory then promises, for every Ok book β :

1. *Confluence*: strong reduction is Church-Rosser, so $\simeq$ is a sensible equality.
2. *Subject reduction*: if $\Gamma \vdash_q t : T \triangleright \pi$ with $q \neq \omega$ and $t \rightarrow t'$ by a weak step, then $\Gamma \vdash_q t' : T \triangleright \pi'$ with $\pi' \leq \pi$: a step never grows the affine measure, and never wakes an erased variable.

3. *Progress*: a closed term well-typed at a live demand is a value or takes a weak step.
4. *Normalization*: a closed term with $\vdash_1 t : T \triangleright \pi$ reduces weakly to a value of its type.
5. *Consistency*: for any family with no constructors, there is no closed t with $\vdash_1 t : \text{Empty} \triangleright \pi$.

The hypotheses are exact, and the mechanization proves it. At demand 0 the calculus deliberately inhabits empty families (erased code is specification, Section 2.4), so claim (5)’s live demand is necessary, and the boundary is itself a theorem (Section 5). Progress is claimed at live demand because an axiom is stuck by design: dead code may rest on an unfilled assert. Strong reduction is not strongly normalizing even on live well-typed values (a value may step inside itself forever), so claim (4)’s weak reachability is the strongest normalization statement that is true, and claim (2) draws the same line: a β under a binder can duplicate the pending call of an affine variable (Section 7), so preservation holds at weak and fails at strong. Terms whose type is the sort are types, and the type level may diverge by design.

3 Why It Terminates

In a functional language there are only two ways to loop: self-application and recursion. BendTT closes them independently, and the two walls are simple enough to state in a paragraph each.

3.1 Self-Application Dies at the Usage Check

Consider $\omega = (\lambda x. xx) (\lambda x. xx)$. The body xx uses x twice, so its measure saturates to ω ; but no binder admits a measure of ω , and the function-type former refuses the quantity outright, so the term is unwritable at *any* type, at any live demand. There is no side-channel to guard: where the predecessor design had to argue that its copyable class excluded functions, this kernel has nothing to exclude, because nothing copies. Curry’s paradox through a negative recursive type [16], and Girard’s paradox through impredicativity [23, 28], both bottleneck through duplicating a $\forall$ -typed value, and die at the same check. Neither a universe hierarchy nor a positivity check is needed.

The point is sharpest at a negative type. The declaration $\text{Trm} : \text{Lam}\{f : \text{Trm} \rightarrow^1 \text{Trm}\}$ is a legal family, the classical paradox vehicle, and a higher-order-abstract-syntax embedding over it is a perfectly fine book: by the claims it is a consistent, terminating development. The paradox engine must apply $\lambda x. \dots x \dots x \dots$ to itself, needs its binder twice, and dies. Nothing guards the negativity except affinity, and the mechanization exhibits exactly this: a well-formed book with a negative type in which Curry’s self-application term typechecks *dead* at an empty family, and provably never live (Section 5).

3.2 Recursion Dies at the Descent

Every live self-call repeats its columns until one argument sits strictly inside its own pattern, one constructor peel deeper. Constructor values in the live fragment are finite

trees, so the lexicographic order on peels is well-founded and the chain of self-calls bottoms out; erased columns, which escape the usage wall, are skipped by the descent for the same reason they escape it: they can never reach live code. Even a HOAS-style loop, $\text{loop}(C\{f\}) = \text{loop}(f(C\{f\}))$, fails: the argument of the self-call is an application, not a subterm of the pattern. And because a book is ordered and a body may not reference a later name, mutual recursion cannot bypass the wall by splitting a loop across two definitions.

Together: no live term can duplicate anything, every recursive call shrinks its own case tree’s columns, and erased code, which escapes both walls, is never consumed at a live demand. Hence every closed live term normalizes and no empty family is inhabited live. Section 5 makes this argument formal; the informal version above is faithful to it.

3.3 What May Diverge

The *dead level* is exempt on purpose. Types may loop; erased positions may loop; so conversion, and with it the checker, is a semi-decision procedure, as conversion checking in dependent type theories already is [15]. A non-halting check reads as “inconclusive”, never as acceptance: the stance is soundness-on-accept, not checker totality. Consistency requires terminating *proofs*, not a terminating checker; NuPRL ran divergent computation inside a consistent logic in the 1980s [14], and Zombie/Trellys combined a consistent fragment with a partial one in a single language [11]. BendTT draws the line at the demand: live terms terminate, dead terms and types need not.

4 The Copiable License

The kernel forbids contraction; programs still need it, an inductive proof feeds one field to the induction hypothesis and a lemma at once. Bend re-admits it as a *license the program proves*. Two definitions, written in Bend itself, state what copying means:

```
def Copy(A, x):
  &y: A -> {x == y : A}

def Copiable(T):
  @x: T -> Copy(T, x) & Copy(T, x)
```

A Copy of x is a value equal to it, packaged with its proof; a $\text{Copiable}(T)$ is a function producing *two* such copies of any value of T . The license enters through one new type former, the Cop type $+T \sim c$, well-formed exactly when c checks against $\text{Copiable}(T)$, and convertible-equal to T itself: the witness is erased and the values are ordinary values of T . A binder of quantity ω (surface $+x$) is permitted if and only if its type is a Cop. The sugar $+T$ derives the witness structurally, $+Nat$ is $+Nat \sim Nat.\text{copy}$, and the base library proves the witnesses for its own first-order types, each copy returning its equality proofs.

Three mechanisms keep the license honest. First, the *locks*: Sigma , Copy and Copiable are locked declarations, the

book validator checks any local spelling of them by conversion against the reference definitions, so a forged Copiable , one that answers two copies without proving them equal to the original, cannot exist under those names, and a closed live proof of an equation normalizes to $\{=\}$. Second, the *promotion rule*: a witness sits in an erased position, but a term checked dead at a Copiable goal is re-checked *live*, its context uses metered against live binders or erased Copiable hypotheses only, and the measured uses are then discarded, the witness stays erased. A dead hypothesis never licenses: the promoted check is what stops an inhabitant of the erased fragment, which can prove anything, from smuggling a copying permit into live code. Third, the standing walls carry the rest: a circular witness dies by descent, an unfilled assert by the live-reference rule, and a contracting inline lambda by the usage cap itself, which never switches off.

The license is an implementation extension, not a kernel rule, and it is stated here with that honesty: the mechanization of Section 5 covers the affine kernel, in which the license does not exist, and the argument for the extension is elementary rather than mechanized: a Copiable witness is a live function that already produces two provably equal copies, so consuming a licensed binder n times abbreviates $n - 1$ witness calls the program could have written by hand. Mechanizing that elaboration is planned work, and the license’s implementation is under active audit. No escape hatch stands beside it: an earlier pragma let a file opt out of the Cop condition and the descent, and it was removed once the whole corpus checked without it; the affine usage cap had never been part of the trade.

5 The Mechanization

The metatheory is mechanized in a single Lean 4 file, `bend2/bend.lean` in the Bend repository, checked with a plain lean invocation: no libraries, no build system. The file has two parts. PART I (under a thousand lines) is the specification, and is the part a human should read: quantities, syntax (de Bruijn), reduction at two strengths, typing, descent, Ok, and the five claims stated as propositions, mirroring the implementation’s own core section by section; Section 2 restates it in mathematical notation. PART II (~19k lines) proves the claims. There are no `sorry`s and the file declares no axioms of its own.

The techniques are conventional where they can be. Confluence is Takahashi’s parallel reduction and complete developments [43]. Subject reduction goes through generation and substitution lemmas with the usage accounting threaded through; the “ ω is never a demand” discipline of Section 2.4 is what makes the substitution lemma true, and the theorem is sharper than preservation alone: the affine measure never grows along a weak step. Normalization and consistency are proven *in full, recursive definitions included*, by a hand-rolled well-founded measure: a Dershowitz–Manna multiset [17] of *charges*, each pricing one pending reference by a per-column size tuple plus a slack slot, paired

Claim	Theorem
Confluence	<code>church_rosser_holds</code>
Subject reduction (weak, measure $\leq$)	<code>subject_reduction_holds</code>
Progress (live)	<code>progress_holds</code>
Normalization (weak, live, typed value)	<code>normalization_holds</code>
Consistency (live)	<code>consistency_holds</code>
The dead boundary, as a witness	<code>consistency_none_boundary</code>

Table 1. The claims and their theorems in `bend.lean`.

with the weight of a typed live erasure of the term. Ordinary interactions keep the charges and strictly drop the weight, the affine beta lands at most one live copy; spending a reference runs its case tree against the call’s arguments, and the descent columns reprise every self-call strictly below the spent tuple, the slack slot paying for underapplied suspensions. Consistency is normalization plus canonical forms at an empty family.

The boundaries of the claims are witnesses, not caveats: the file exhibits a well-formed book with a negative recursive type in which Curry’s ω term checks *dead* at an empty family (`dead_omega_check`, packaged as `consistency_none_boundary`), which is exactly why claims (4) and (5) demand liveness.

Provenance, stated plainly: the specification was written and reviewed by humans; the proofs of PART II were written by an AI system (Claude Fable 5, Anthropic) and are checked by Lean. The humans audited the statements, not the proof bodies. We consider this trust story acceptable because it is the one proof assistants offer in general: trust the checker, audit the statements.

6 From BendTT to Bend

The reference implementation is a bidirectional checker [18, 37] for this discipline, plus surface features. The differences from the calculus are engineering, not logic; we list them, and the ones that are genuine extensions are flagged as unformalized.

Patterns and motives. Bend’s `match/case` blocks flatten into case trees of the one-constructor peel [5, 33]: first row wins, uncovered cases become the empty match, and exhaustiveness is the checker’s concern, not the parser’s, an unreachable hole is exactly a well-typed empty match. Because there is no unification, a match whose motive genuinely depends on its scrutinees is written *framed*: the source states the motive, and the flattener rebuilds the tree annotated with it. This is a design commitment, not a gap: every feature known to make checking slow or unpredictable, metavariables, implicit arguments, global search, was left out, because Bend aims at checking large codebases at ordinary-compiler speeds.

Binders and elaboration. The checker represents binder bodies as host-language closures (higher-order abstract syntax [13, 36]): substitution is application in the host, and no de Bruijn machinery exists outside the formalization. Combined with conversion by `reduce-and-compare`, the checker is in effect an evaluator in the style of normalization by evaluation [7]. It elaborates as it checks, annotating every layer; the elaborated body is stored on the definition and consumed by the compiler, never re-consumed by the checker itself.

Asserts, fills, and the world. The `assert/fill` split of Section 2.1 is the surface’s own declaration form, and it carries the language’s one doorway to effects: a fill whose body is a list of foreign imports is an effectful primitive, allowed only at the base library’s `I0` type and run only by the event loop, so an axiom can be stuck and still name an effect the runtime performs. The typed interface stays honest: a foreign name is stuck for conversion, live-usable for the event loop, and nothing else.

Unformalized extensions. Machine 32-bit words and floats (the float family is axiomatic in the base library), flat arrays with in-place backends, the base library’s native representations, and the Copiable license of Section 4, whose kernel-side meaning is the elaboration argument given there. The compiler is bound by a cost law the checker sets: it may drop a cost the source spelled, a licensed copy may become a counted share or a borrow, but it may never add a clone or a retain the source did not spell [42]. None of these features interacts with the consistency argument, and formalizing the license is the next planned extension of the mechanization.

7 Discussion

7.1 What Affinity Takes Away

The honest price is contraction on closures. The standard map : $\forall^1 f : (A \rightarrow^1 B). \forall^1 xs : \text{List } A. \text{List } B$

is ill-typed: f is used once per element, its binder would need ω , and no Copiable witness for a function type can exist, a copy would have to prove its clone equal to the original, which intensional equality does not grant for closures. There is no workaround that smuggles a closure copy in. The practical escape is that *code* is free: a top-level definition may be referenced any number of times, so specialized maps, or maps over licensed data in place of closures, cover the common idioms; but first-class closure-heavy style does not transfer. Bend accepts this deliberately: the restriction is also what the runtime wants [42], and years of writing in this discipline suggest the loss is bearable.

7.2 Expressiveness

A consistency proof means little if little can be said under it, so we state the reach directly. On the program side the baseline is C: first-order data, machine words, arrays updated in place, top-level code called by name. That fragment passes through affinity untouched, and it is the one scripts, servers,

games and operating systems are written in, none of which ever needed a copied closure. What does not transfer is a style, the Haskell one, not a class of programs.

On the proof side the freedom is larger, not smaller. Statements are erased: at demand 0 occurrences cost nothing, so a theorem may quantify over functions, repeat variables, and apply anything freely; affinity constrains what runs, never what is said. Live proofs draw their duplication from the license at first-order data, which is what induction wants: the base library proves its own copy witnesses, each returning equality proofs, derives its equality kit (congruence, symmetry, transitivity, each a single rewrite), and proves commutativity of addition for both unary naturals and 32-bit words, the successor field feeding the induction hypothesis and a lemma at once, checked by Bend itself. Theorems whose subjects are first-order, most of mathematics, never meet the restriction at all.

7.3 Anticipated Objections

“This is just QTT.” The quantities come from McBride and Atkey [4, 34], and the restriction of demands to $\{0, 1\}$ mirrors Atkey’s. But QTT admits ω binders at every type, so QTT with $\text{Type} : \text{Type}$ is Girard-inconsistent; its consistency comes from the universe hierarchy it keeps. BendTT has no ω binders at all, no scaling and no semiring, and the copying permit is a program-level proof (Section 4): the absence carries the entire weight the hierarchy would.

“Equality needs non-linear variables: $\text{refl} : \forall a. \{a = a : T\}$ mentions a twice.” Occurrences in types cost nothing; only live demands count. $\text{refl} = \lambda a. \{=\}$ checks with an erased binder: the equation’s components are erased, and the proof term carries nothing. Rewriting with an equation consumes the equation once, like any other value.

“Without positivity you get Curry’s paradox.” The Curry term exists and typechecks *dead*, the mechanization exhibits it at an empty family in a well-formed book with a negative type (`consistency_none_boundary`), but no live judgment accepts it: the duplication it needs is unwritable. The boundary is a theorem rather than an apology.

“Reduction can break affinity under a λ .” Yes: a strong step under a binder can duplicate the *pending call* of an affine variable by substituting it into a body that repeats its own binder in a dead position. No evaluator takes such a step: checker and runtime evaluate weakly, and claims (2) and (4) are stated at weak reduction for this reason; consistency asks nothing more, since evaluating a closed program only fires closed redexes.

“The checker can loop.” Yes, on dead code, by design (Section 3.3). A hang accepts nothing, and the deployed checker is deterministic, so the same program is admitted or rejected identically on every machine.

“Erased code can inhabit Empty is that not inconsistency?” No: erased code is specification. Nothing live can consume an erased inhabitant, no rule coerces dead to live, and the theorems quantify over live judgments. The mechanization states this boundary as a theorem rather than hiding it.

“A forged Copiable could license anything.” It could, which is why the three names it depends on are locked against their reference definitions by the book validator, why the witness position is re-checked live under the promotion rule, and why the license lives outside the kernel: the mechanized claims do not rest on it (Section 4).

“Why not just use universes?” Universes are the orthodox road, and nothing prevents adding a hierarchy to BendTT later. But Bend needs affinity anyway, for its runtime; reusing it for consistency means one mechanism where there would otherwise be two, plus level arithmetic, cumulativity, and universe polymorphism. $\text{Type} : \text{Type}$ also buys real expressivity: impredicative encodings and type-computing definitions that no predicative hierarchy accepts.

7.4 Limitations

The consistency result is syntactic: normalization plus canonical forms, formalized in Lean. It therefore holds relative to the consistency of Lean’s own foundation. There is no semantic model: no set-theoretic, denotational or realizability interpretation, and no conservativity result over an established theory. A model would explain *why* affinity carries the weight, not only that it does; realizability for linear dependent theories [40] looks like a starting point, and this is future work. Conversion is joinability of reduction with η for functions and nothing for pairs. The Copiable license is argued, locked and audited, but not yet mechanized (Section 4). And the theorems are about the calculus, not the code: the reference checker is ordinary unverified software, and Section 6’s correspondence is an engineering claim, not a theorem.

7.5 Related Work

Linear-plus-dependent systems go back to LLF [12] and continue through [20, 30, 32, 45]; the quantitative line [4, 8, 34, 35] is the closest in mechanism, and Abel et al. mechanized a graded theory with a predicative universe [2]. To our knowledge none of these uses the absence of contraction for consistency: a 2023 linear dependent type theory still adds universe levels “to avoid Girard’s paradox” [21], and recent work on impredicativity in linear dependent theories obtains an impredicative universe, not $\text{Type} : \text{Type}$, via a realizability model [40]. The consistent-logic-with-partiality line is [11, 14, 46], and per-definition termination certificates are ACL2’s definitional principle [29]. Mechanized metatheories of practical kernels include [3, 10, 39]; ours is smaller in scope but proves normalization rather than assuming it. The small-kernel tradition runs from Automath [6, 9] through LCF and HOL Light [25, 27]; Bend’s predecessor kernels in the same ecosystem were based on self types [19, 41].

8 Conclusion

BendTT buys consistency with affinity instead of a universe hierarchy, checks recursion with a descent rule simple enough to audit in an afternoon, and is mechanized end to end, recursive definitions included. The one liberty its pre-

decessor took inside the kernel, a built-in class of copyable types, has been evicted: the kernel now refuses all duplication, and the surface earns it back with proofs. Nothing in the calculus is difficult, and that is the point: it is meant to be read, audited, and extended. The mechanization will keep growing toward the implemented language, the copying license first, and perhaps toward stronger equalities: we watch the higher observational line, Narya in particular [38], as the likely shape of that redesign.

Acknowledgments

The proofs in `bend.lean` were written by Claude Fable 5 (Anthropic) and checked by Lean 4; this text was drafted with the same model. Thanks to the Higher Order Company team for discussions.

References

- [1] Andreas Abel and Thorsten Altenkirch. 2002. A Predicative Analysis of Structural Recursion. *Journal of Functional Programming* 12, 1 (2002), 1–41.
- [2] Andreas Abel, Nils Anders Danielsson, and Oskar Eriksson. 2023. A Graded Modal Dependent Type Theory with a Universe and Erasure, Formalized. *Proceedings of the ACM on Programming Languages* 7, ICFP (2023), 220:1–220:35.
- [3] Andreas Abel, Joakim Öhman, and Andrea Vezzosi. 2018. Decidability of Conversion for Type Theory in Type Theory. *Proceedings of the ACM on Programming Languages* 2, POPL (2018), 23:1–23:29.
- [4] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2018, 56–65.
- [5] Lennart Augustsson. 1985. Compiling Pattern Matching. In *Functional Programming Languages and Computer Architecture (FPCA)* (LNCS), 1985. Springer, 368–381.
- [6] Henk Barendregt and Herman Geuvers. 2001. Proof-Assistants Using Dependent Type Systems. *Handbook of Automated Reasoning*, 1149–1238.
- [7] Ulrich Berger and Helmut Schwichtenberg. 1991. An Inverse of the Evaluation Functional for Typed λ -Calculus. In *Proceedings of the Sixth Annual IEEE Symposium on Logic in Computer Science (LICS)*, 1991, 203–211.
- [8] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. In *35th European Conference on Object-Oriented Programming (ECOOP)* (LIPIcs), 2021, 9:1–9:26.
- [9] Nicolaas G. de Bruijn. 1980. A Survey of the Project Automath. To *H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, 579–606.
- [10] Mario Carneiro. 2024. Lean4Lean: Towards a Formalized Metatheory for the Lean Theorem Prover. Retrieved from <https://arxiv.org/abs/2403.14064>
- [11] Chris Casinghino, Vilhelm Sjöberg, and Stephanie Weirich. 2014. Combining Proofs and Programs in a Dependent Typed Language. In *Proceedings of the 41st ACM Symposium on Principles of Programming Languages (POPL)*, 2014, 33–45.
- [12] Iliano Cervesato and Frank Pfenning. 2002. A Linear Logical Framework. *Information and Computation* 179, 1 (2002), 19–75.
- [13] Adam Chlipala. 2008. Parametric Higher-Order Abstract Syntax for Mechanized Semantics. In *Proceedings of the 13th ACM International Conference on Functional Programming (ICFP)*, 2008, 143–156.
- [14] Robert L. Constable and Scott F. Smith. 1987. Partial Objects in Constructive Type Theory. In *Proceedings of the Second Annual IEEE Symposium on Logic in Computer Science (LICS)*, 1987, 183–193.
- [15] Thierry Coquand. 1991. An Algorithm for Testing Conversion in Type Theory. *Logical Frameworks*, 255–279.
- [16] Haskell B. Curry. 1942. The Inconsistency of Certain Formal Logics. *Journal of Symbolic Logic* 7, 3 (1942), 115–117.
- [17] Nachum Dershowitz and Zohar Manna. 1979. Proving Termination with Multiset Orderings. *Communications of the ACM* 22, 8 (1979), 465–476.
- [18] Jana Dunfield and Neel Krishnaswami. 2021. Bidirectional Typing. *ACM Computing Surveys* 54, 5 (2021), 98:1–98:38.
- [19] Peng Fu and Aaron Stump. 2014. Self Types for Dependently Typed Lambda Encodings. In *Rewriting and Typed Lambda Calculi (RTA-TLCA)* (LNCS), 2014. Springer, 224–239.
- [20] Peng Fu, Kohei Kishida, and Peter Selinger. 2020. Linear Dependent Type Theory for Quantum Programming Languages. In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2020, 440–453.
- [21] Qiancheng Fu and Hongwei Xi. 2023. A Two-Level Linear Dependent Type Theory. Retrieved from <https://arxiv.org/abs/2309.08673>
- [22] Eduardo Giménez. 1994. Codifying Guarded Definitions with Recursive Schemes. In *Types for Proofs and Programs (TYPES)* (LNCS), 1994. Springer, 39–59.
- [23] Jean-Yves Girard. 1972. Interprétation fonctionnelle et élimination des coupures de l’arithmétique d’ordre supérieur. Doctoral dissertation.
- [24] Jean-Yves Girard. 1998. Light Linear Logic. *Information and Computation* 143, 2 (1998), 175–204.
- [25] Michael J. C. Gordon, Robin Milner, and Christopher P. Wadsworth. 1979. *Edinburgh LCF: A Mechanised Logic of Computation*. Springer.
- [26] V. N. Grišin. 1982. Predicate and Set-Theoretic Calculi Based on Logic Without Contractions. *Mathematics of the USSR–Izvestiya* 18, 1 (1982), 41–59.
- [27] John Harrison. 2009. HOL Light: An Overview. In *Theorem Proving in Higher Order Logics (TPHOLs)* (LNCS), 2009. Springer, 60–66.
- [28] Antonius J. C. Hurkens. 1995. A Simplification of Girard’s Paradox. In *Typed Lambda Calculi and Applications (TLCA)* (LNCS), 1995. Springer, 266–278.
- [29] Matt Kaufmann, Panagiotis Manolios, and J Strother Moore. 2000. *Computer-Aided Reasoning: An Approach*. Kluwer Academic Publishers.
- [30] Neelakantan R. Krishnaswami, Pierre Pradic, and Nick Benton. 2015. Integrating Linear and Dependent Types. In *Proceedings of the 42nd ACM Symposium on Principles of Programming Languages (POPL)*, 2015, 17–30.
- [31] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. 2001. The Size-Change Principle for Program Termination. In *Proceedings of the 28th ACM Symposium on Principles of Programming Languages (POPL)*, 2001, 81–92.
- [32] Zhaohui Luo and Yu Zhang. 2016. A Linear Dependent Type Theory. In *Types for Proofs and Programs (TYPES)*, Novi Sad (abstract), 2016.
- [33] Luc Maranget. 2008. Compiling Pattern Matching to Good Decision Trees. In *Proceedings of the ACM SIGPLAN Workshop on ML*, 2008, 35–46.
- [34] Conor McBride. 2016. I Got Plenty o’ Nuttin’. *A List of Successes That Can Change the World 9600*, 207–233.
- [35] Benjamin Moon, Harley Eades III, and Dominic Orchard. 2021. Graded Modal Dependent Type Theory. In *European Symposium on Programming (ESOP)* (LNCS), 2021. Springer, 462–490.
- [36] Frank Pfenning and Conal Elliott. 1988. Higher-Order Abstract Syntax. In *Proceedings of the ACM Conference on Programming Language Design and Implementation (PLDI)*, 1988, 199–208.
- [37] Benjamin C. Pierce and David N. Turner. 2000. Local Type Inference. *ACM Transactions on Programming Languages and Systems* 22, 1 (2000), 1–44.
- [38] Michael Shulman. 2024. Narya: A Proof Assistant for Higher Observational Type Theory. Retrieved from <https://github.com/mikeshulman/narya>
- [39] Matthieu Sozeau, Simon Boulrier, Yannick Forster, Nicolas Tabareau, and Théo Winterhalter. 2020. Coq Coq Correct! Verification of Type Checking and Erasure for Coq, in Coq. *Proceedings of the ACM on Programming Languages* 4, POPL (2020), 8:1–8:28.
- [40] Sam Speight and Niels van der Weide. 2026. Impredicativity in Linear Dependent Type Theory. Retrieved from <https://arxiv.org/abs/2602.08846>
- [41] Aaron Stump. 2017. The Calculus of Dependent Lambda Eliminations. *Journal of Functional Programming* 27, (2017), e14.
- [42] Victor Taelin. 2026. *BendRT: A Parallel Runtime for CPUs and GPUs*.

- [43] Masako Takahashi. 1995. Parallel Reductions in λ -Calculus. *Information and Computation* 118, 1 (1995), 120–127.
- [44] Kazushige Terui. 2004. Light Affine Set Theory: A Naive Set Theory of Polynomial Time. *Studia Logica* 77, (2004), 9–40.
- [45] Matthijs Vákár. 2015. A Categorical Semantics for Linear Logical Frameworks. In *Foundations of Software Science and Computation Structures (FoSSaCS) (LNCS)*, 2015. Springer.
- [46] Stephanie Weirich, Antoine Voizard, Pedro Henrique Azevedo de Amorim, and Richard A. Eisenberg. 2017. A Specification for Dependent Types in Haskell. *Proceedings of the ACM on Programming Languages* 1, ICFP (2017), 31:1–31:29.